

AULA 06 – PONTEIROS E SUAS OPERAÇÕES

ROTEIRO

- PONTEIROS
- SINTAXE EM C
- OPERAÇÕES
- VETOR DE PONTEIRO
- APLICAÇÃO



INTRODUÇÃO

- Para que um programa de computador possa manipular dados durante a sua execução é necessário que os mesmos estejam armazenados na memória principal (também conhecida como memória RAM).
- Cada variável de um programa ocupa um certo número de bytes consecutivos na memória do computador. Uma variável do tipo char ocupa 1 byte. Uma variável do tipo int ocupa 4 bytes e um double ocupa 8 bytes em muitos computadores.

INTRODUÇÃO

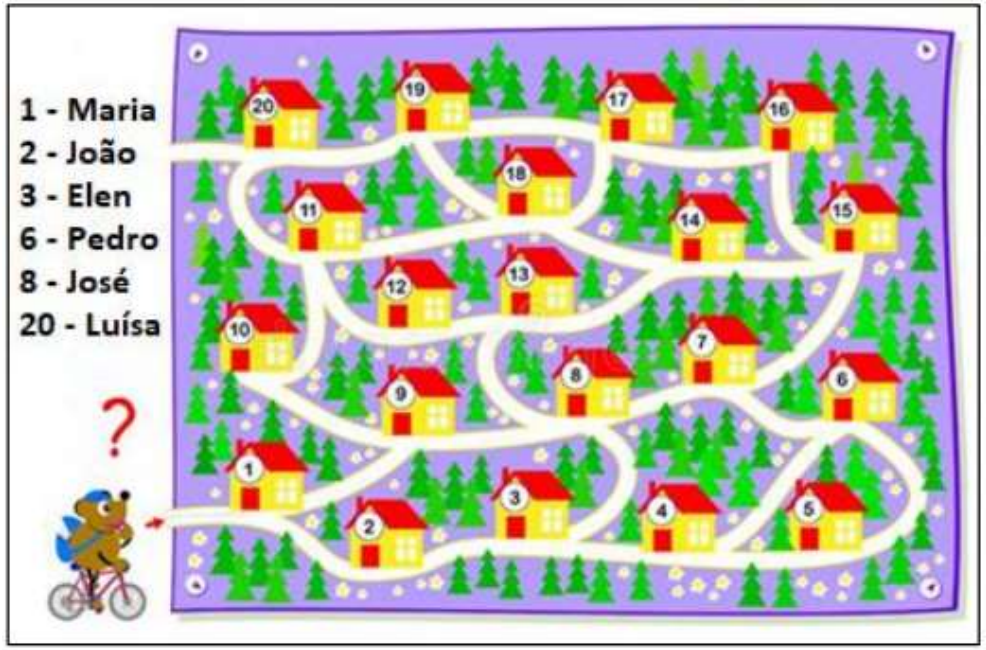
- O número exato de bytes de uma variável é dado pelo operador sizeof. A expressão sizeof (char), por exemplo, vale 1 em todos os computadores e a expressão sizeof (int) vale 4 em muitos computadores.
- Cada variável (em particular, cada registro e cada vetor) na memória tem um endereço. Na maioria dos computadores, o endereço de uma variável é o endereço do seu primeiro byte.

INTRODUÇÃO

- Analogamente, nós podemos comparar a memória de um computador com uma rua. Ela possui uma quantidade de casas (posições), cada qual com o seu endereço (número) e que permite uma capacidade máxima de pessoas (conteúdo a ser armazenado).
- Agora imagine que um carteiro realiza o papel do computador, ele deve localizar o endereço correto para entregar uma carta ou encomenda.

INTRODUÇÃO

- Para que o carteiro possa localizar a casa com facilidade, é necessário que as residências estejam numeradas em uma ordem lógica.



INTRODUÇÃO

- O endereço de uma variável é dado pelo operador & . Assim, se i é uma variável então &i é o seu endereço. No exemplo abaixo, &i vale 89422 e &c vale 89421.

Variável	Endereço
C	89421
I	89422
PONTO	89426
V[0]	89434
V[1]	89438
V[2]	89442

PONTEIROS

- Suponha a seguinte situação-problema: você deseja ir até a casa de um amigo e possui apenas um papel com o endereço desse amigo.



PONTEIROS

- Quando você anota o endereço de um amigo no papel, você está criando um ponteiro.
- Qual o sentido disto? Simples! Quando você anota o endereço de um colega, você pode utilizar essa informação para achar a rota que você deve seguir para achá-lo. A linguagem C funciona assim com relação ao funcionamento de ponteiros.

PONTEIROS

- Um ponteiro é um tipo especial de variável que armazena um endereço. Um ponteiro pode ter o valor NULL indicando que é um endereço “inválido”.
- Se um ponteiro p armazena o endereço de uma variável i, podemos dizer “p aponta para i” ou “p é o endereço de i”. Se um ponteiro p tem o valor diferente de NULL então *p é o valor da variável apontada por p.

PONTEIROS

- Do lado esquerdo, um ponteiro p, armazenado no endereço 60001, contém o endereço de um inteiro. O lado direito dá uma representação esquemática da situação:



PONTEIROS EM C

- Na linguagem C, é necessário informar ao compilador para que tipo de variável, o ponteiro declarado irá apontar.
  - Um ponteiro int aponta para uma variável do tipo inteiro, isto é, guarda o endereço (posição de memória) de um número inteiro.
  - Um ponteiro char aponta para uma variável do tipo caractere, isto é, guarda o endereço de um caractere, e assim, por diante.

PONTEIROS EM C

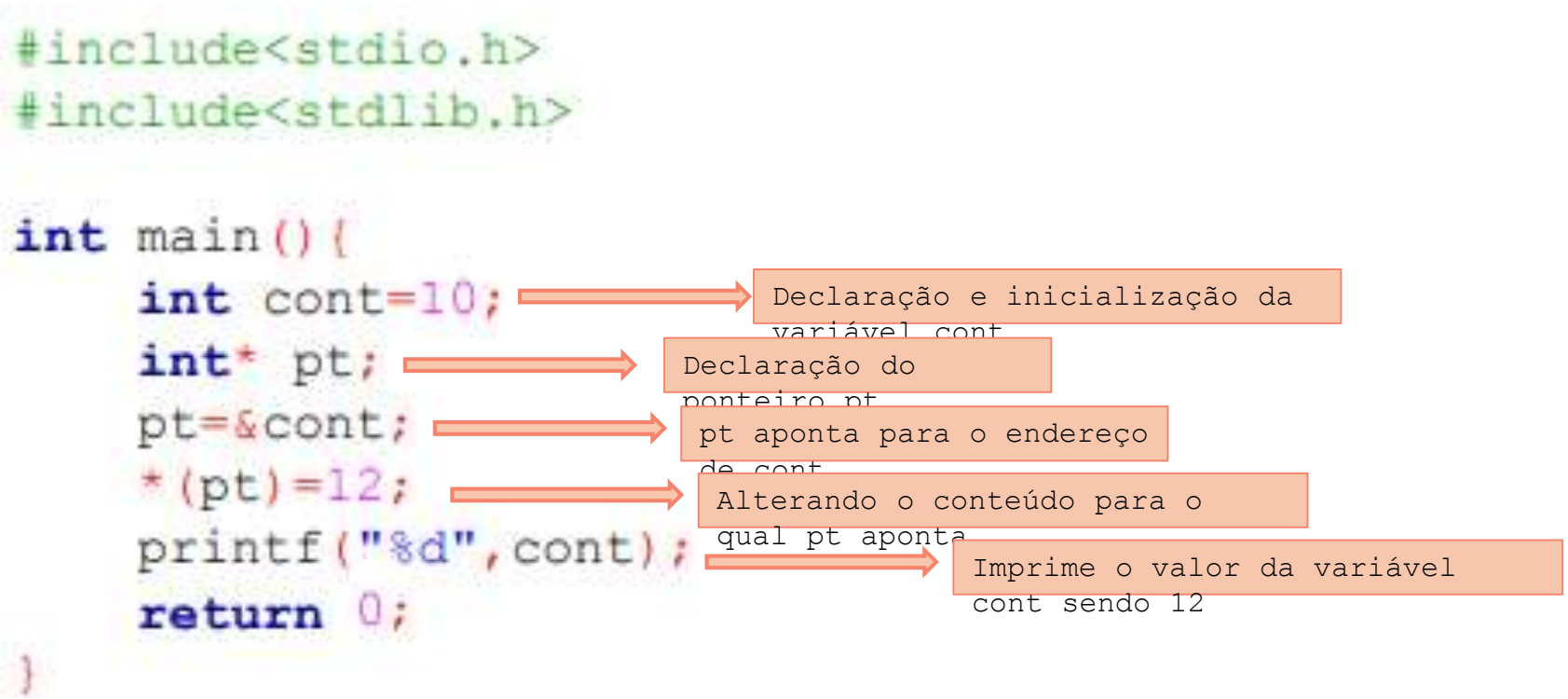
- Para declarar um ponteiro em C:

```c
int main(){
    //Declara uma variável int contendo o valor 10
    int c = 10;
    //Declara um ponteiro para int
    int *p;
    //Atribui ao ponteiro o endereço da variável int
    p = &c ;

    return 0;
}
```

PONTEIROS EM C

- Manipulando valores com ponteiro em C:



UTILIZAÇÃO DE PONTEIROS

- A utilização de ponteiros permite uma manipulação mais eficiente em estruturas de dados, principalmente estruturas dinâmicas, tais como: pilha, lista, fila, árvores e grafos. Pois, a alocação de espaços de memória pode ser realizada durante a execução do programa.
- Uma desvantagem é o aumento da complexidade do entendimento e implementação dos algoritmos.

OPERAÇÕES COM PONTEIROS

- Operações válidas entre ponteiros:

Operação	Descrição
p1=p2	p1 e p2 irão apontar para o mesmo local
*p1=*p2	O conteúdo de p1 passa a ser o mesmo conteúdo apontado por p2
p1++	Adiciona 1 no valor do endereço para o qual p1 aponta
p2--	Subtrai 1 no valor do endereço para o qual p2 aponta
(*p1)++	Adiciona 1 ao conteúdo armazenado no endereço para o qual p1 aponta
(*p2)--	Subtrai 1 ao conteúdo armazenado no endereço para o qual p1 aponta
p1 = p1+15	p1 irá apontar para o endereço que está 15 posições a frente
p2=p2-2	p2 irá apontar para o endereço que está 2 posições atrás
*(p1+15)	Será mostrado o conteúdo armazenado 15 posições a frente do endereço apontado por p1
*(p2-5)	Será mostrado o conteúdo armazenado 2 posições atrás do endereço

OPERAÇÕES COM PONTEIROS

- É possível ainda realizar comparação entre ponteiros:
  - p1 == p2
  - p1 != p2
  - p1 > p2
  - p1 < p2

VETORES COMO PONTEIROS

- Quando você declara um vetor em linguagem C, o compilador calcula o tamanho, em bytes, necessário para armazenar o vetor.
- Esse cálculo é realizado multiplicando o valor do tamanho do vetor pelo tamanho do tipo de dados do vetor. O compilador então aloca este número de bytes em um espaço livre de memória.

VETORES COMO PONTEIROS

- O nome do vetor que você declarou é na verdade um ponteiro para o tipo de dados do vetor. Depois de alocado na memória o espaço para o vetor, o compilador toma o nome da variável (que é um ponteiro) e aponta para o primeiro elemento do vetor.
  - vet[indice] é equivalente a *(vet+indice).

VETORES COMO PONTEIROS

```
int *ponteiro, i, vetor[3];
 ponteiro = &i;

/*as operacoes a seguir sao invalidas */
 vetor = vetor + 2;     /* ERRADO */
 vetor++;               /* ERRADO */
 vetor = ponteiro;      /* ERRADO */

 /* as operacoes abaixo sao validas */
 ponteiro = vetor;       /* CERTO */
 ponteiro = vetor+2;     /* CERTO */
```

PRATIQUE

- **Exercício 01:** Escreva um programa que declare um vetor de inteiros com 100 posições. Você deve inicializar o vetor com zeros usando ponteiros para endereçar seus elementos. Preencha depois o vetor com os números de 1 a 100, também usando ponteiros.

PRATIQUE

- **Exercício 02:** Sendo p um ponteiro, qual a diferença entre os comandos apresentados abaixo:
  - p++:
  - (*p)++:
  - *(p++):
  - *(p+10):

PRATIQUE

- **Exercício 03:** Qual o valor de y no final do programa abaixo?

```
int main() {

int y, *p, x;
y = 0;
p = &y;
x = *p;
x = 4;
(*p)++;
x--;
(*p) += x;
 printf ("y = %d\n", y);
 return(0);
}
```

DÚVIDAS?

